> *give me a complete guide for the project for "Workaround 2: Use Wokwi for VS Code (Open-Source Extension)"*

This guide covers setting up, compiling, simulating, and interacting with the ESP32 NAS and Static Web Server locally using the **Wokwi for VS Code** extension.

By running the simulation locally on your workstation, the ESP32 runs via your local toolchain, and network requests can be bridged directly to your host loopback adapter without cloud paywalls.

Prerequisites & Software Installation

1. **Visual Studio Code:** Download and install VS Code.
2. **VS Code Extensions:**

- Open the Extensions panel (`Ctrl + Shift + X`).
- Search for and install **Wokwi Simulator** (by Wokwi).
- Search for and install **PlatformIO IDE** (by PlatformIO) or the **Arduino** extension. *(PlatformIO is strongly recommended for deterministic library management and compilation)*.

Step 1: Project Directory Structure

Create a dedicated folder for your project on your PC (e.g., `ESP32_NAS_Server`) and structure it as follows:

```
ESP32_NAS_Server/
│
├── platformio.ini         # Toolchain & dependency configuration
├── wokwi.toml             # Wokwi simulation descriptor
├── diagram.json           # Circuit schematics and pin mappings
│
└── src/
    └── main.cpp           # Firmware code (formerly sketch.ino)
```

Step 2: Configure `platformio.ini`

Create `platformio.ini` in the project root. This tells PlatformIO to compile your C++ firmware for the ESP32 DevKit using the ESP32 Arduino Core:

```
[env:esp32dev]
platform = espressif32
board = esp32dev
framework = arduino
monitor_speed = 115200

; Built-in libraries used directly: WiFi, WebServer, SPI, SD, FS
lib_deps =
```

Step 3: Configure `wokwi.toml`

Create `wokwi.toml` in your root folder. This instructs the local Wokwi extension on how to run your compiled firmware:

```
[wokwi]
version = 1
firmware = ".pio/build/esp32dev/firmware.bin"
elf = ".pio/build/esp32dev/firmware.elf"
```

Step 4: Configure Circuit Schematics (`diagram.json`)

Create `diagram.json` in your root directory. This configures the virtual ESP32 and MicroSD Card wired across the standard SPI pins:

```
{
  "version": 1,
  "author": "Gaus Saraf Murady",
  "editor": "wokwi",
  "parts": [
    { "type": "wokwi-esp32-devkit-v1", "id": "esp", "top": 0, "left": 0, "attrs": {} },
    { "type": "wokwi-microsd-card", "id": "sd1", "top": 0, "left": 220, "attrs": {} }
  ],
```

```
  "connections": [
    [ "esp:TX0", "$serialMonitor:RX", "", [] ],
    [ "esp:RX0", "$serialMonitor:TX", "", [] ],
    [ "esp:3V3", "sd1:VCC", "red", [ "v0" ] ],
    [ "esp:GND.1", "sd1:GND", "black", [ "v0" ] ],
    [ "esp:5", "sd1:CS", "yellow", [ "v0" ] ],
    [ "esp:23", "sd1:DI", "blue", [ "v0" ] ],
    [ "esp:19", "sd1:DO", "green", [ "v0" ] ],
    [ "esp:18", "sd1:CLK", "orange", [ "v0" ] ]
  ]
}
```

Step 5: Firmware Code (`src/main.cpp`)

Place the following code inside `src/main.cpp`. It runs the file server, seeds the static web hosting assets to the virtual SD card, and sets up HTTP endpoints:

```
#include <Arduino.h>
#include <WiFi.h>
#include <WebServer.h>
#include <SPI.h>
#include <SD.h>

#define SD_CS_PIN 5

// Wokwi virtual Wi-Fi access point credentials
const char* ssid = "Wokwi-GUEST";
const char* password = "";

WebServer server(80);

String formatBytes(size_t bytes) {
  if (bytes < 1024) return String(bytes) + " B";
  if (bytes < (1024 * 1024)) return String(bytes / 1024.0, 1) + " KB";
  return String(bytes / (1024.0 * 1024.0), 1) + " MB";
}

void ensureInitialFiles() {
  if (!SD.exists("/www")) {
    SD.mkdir("/www");
  }

  if (!SD.exists("/www/index.html")) {
```

```
    File html = SD.open("/www/index.html", FILE_WRITE);
    if (html) {
      html.println("<!DOCTYPE html><html><head><meta charset='UTF-8'>");
      html.println("<title>ESP32 Web Server</title></head>");
      html.println("<body style='font-family:sans-serif; text-align:center; padding:30px; back
ground:#f4f4f4;'>");
      html.println("<h1>Hosted on ESP32 MicroSD Storage</h1>");
      html.println("<p>Dual-Purpose Node: <strong>Static Web Host Active</strong></p>");
      html.println("<p><a href='/'>Go to NAS File Browser</a></p>");
      html.println("</body></html>");
      html.close();
      Serial.println("[STORAGE] Seeded /www/index.html");
    }
  }

  if (!SD.exists("/lab_report.txt")) {
    File report = SD.open("/lab_report.txt", FILE_WRITE);
    if (report) {
      report.println("Course: CSE0611324 Microprocessor Lab");
      report.println("Project: ESP32 Embedded NAS & Web Node");
      report.println("Storage: FAT32 block device over hardware SPI");
      report.close();
      Serial.println("[STORAGE] Seeded /lab_report.txt");
    }
  }
}

// NAS File Browser Route Handler
void handleRoot() {
  uint64_t total = SD.totalBytes();
  uint64_t used = SD.usedBytes();

  String html = "<!DOCTYPE html><html><head><meta charset='UTF-8'>";
  html += "<meta name='viewport' content='width=device-width, initial-scale=1.0'>";
  html += "<title>ESP32 NAS Node</title><style>";
  html += "body { font-family: monospace; background: #121212; color: #e0e0e0; padding: 20px;
}";
  html += ".card { max-width: 650px; margin: auto; background: #1e1e1e; padding: 20px; border-
radius: 8px; }";
  html += "table { width: 100%; border-collapse: collapse; margin-top: 15px; }";
  html += "th, td { border: 1px solid #333; padding: 8px; text-align: left; }";
  html += "th { background: #2a2a2a; }";
  html += "a { color: #64B5F6; text-decoration: none; }";
  html += "a:hover { text-decoration: underline; }";
  html += "</style></head><body><div class='card'>";
  html += "<h2>ESP32 Local Storage NAS</h2>";
  html += "<p><b>Capacity:</b> " + formatBytes(total) + " | <b>Used:</b> " + formatBytes(used)
```

```
+ "</p>";
  html += "<p><a href='/site'>View Hosted Static Website (/site)</a></p>";
  html += "<table><tr><th>File Name</th><th>Size</th><th>Action</th></tr>";

  File root = SD.open("/");
  File file = root.openNextFile();
  while (file) {
    if (!file.isDirectory()) {
      String fname = String(file.name());
      if (fname.startsWith("/")) fname = fname.substring(1);
      html += "<tr><td>" + fname + "</td>";
      html += "<td>" + formatBytes(file.size()) + "</td>";
      html += "<td><a href='/get?file=" + fname + "'>Download</a></td></tr>";
    }
    file = root.openNextFile();
  }
  html += "</table></div></body></html>";

  server.send(200, "text/html", html);
}

// Static Web Host Endpoint
void handleSite() {
  if (SD.exists("/www/index.html")) {
    File htmlFile = SD.open("/www/index.html", FILE_READ);
    server.streamFile(htmlFile, "text/html");
    htmlFile.close();
  } else {
    server.send(404, "text/plain", "Static Site Not Found");
  }
}

// NAS File Stream Endpoint
void handleDownload() {
  if (!server.hasArg("file")) {
    server.send(400, "text/plain", "Missing file parameter");
    return;
  }

  String filename = "/" + server.arg("file");
  if (!SD.exists(filename)) {
    server.send(404, "text/plain", "File Not Found");
    return;
  }

  File downloadFile = SD.open(filename, FILE_READ);
  server.sendHeader("Content-Disposition", "attachment; filename=\"" + server.arg("file") +
```

```
"\"");
  server.streamFile(downloadFile, "application/octet-stream");
  downloadFile.close();
}

void setup() {
  Serial.begin(115200);
  delay(1000);
  Serial.println("\n--- Starting ESP32 NAS & Web Node (VS Code Local) ---");

  // Mount SD Storage
  if (!SD.begin(SD_CS_PIN)) {
    Serial.println("[ERROR] MicroSD card initialization failed!");
    return;
  }
  Serial.printf("[STORAGE] Online. Total Space: %llu MB\n", SD.totalBytes() / (1024 * 1024));
  ensureInitialFiles();

  // Connect to Virtual Wi-Fi
  Serial.printf("[WIFI] Connecting to %s...", ssid);
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(200);
    Serial.print(".");
  }
  Serial.println("\n[WIFI] Connected!");
  Serial.printf("[NET] Node IP Address: %s\n", WiFi.localIP().toString().c_str());

  // Bind Routes
  server.on("/", HTTP_GET, handleRoot);
  server.on("/site", HTTP_GET, handleSite);
  server.on("/get", HTTP_GET, handleDownload);

  server.begin();
  Serial.println("[HTTP] Server listening on port 80.");
}

void loop() {
  server.handleClient();
}
```

Step 6: Build & Run the Simulation

1. **Compile the Firmware:**

- In VS Code, click the **PlatformIO Build checkmark (✓)** on the bottom status bar (or press `Ctrl + Alt + B`).
- PlatformIO will compile your code and produce `.pio/build/esp32dev/firmware.bin`.

2. **Start Wokwi Simulation:**

- Open `diagram.json` in VS Code.
- Press **`F1`** (or `Ctrl + Shift + P`) to open the VS Code Command Palette.
- Type and run: **`Wokwi: Start Simulator`**.
- A simulated interactive circuit preview will launch directly inside a VS Code tab.

3. **Inspect Output:**

- Open the built-in Wokwi Serial Monitor in VS Code.
- Verify that storage mounts, files seed, and the simulated Wi-Fi connects:

```
[STORAGE] Online. Total Space: ...
[WIFI] Connected!
[NET] Node IP Address: 10.0.x.x
[HTTP] Server listening on port 80.
```

Step 7: Bridging Network Traffic Locally

Because the simulation runs locally on your computer via VS Code, you can forward inbound traffic without cloud restriction:

1. Open your terminal in the project directory.
2. Run the standalone wokwigw binary with explicit port forwarding enabled:

- **Windows:**

```
wokwigw.exe --forward 9080:10.13.37.2:80
```

- **Linux/macOS:**

```
./wokwigw --forward 9080:10.13.37.2:80
```

3. Open your desktop web browser (Chrome, Edge, Firefox):

- **NAS Dashboard:** `http://localhost:9080/`
- **Hosted Static Website:** `http://localhost:9080/site`
- **Download Test File:** Click on `lab_report.txt` in the table to stream the file from the virtual SD card directly to your host PC's downloads folder.